



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

LABORATORIO SESIÓN 7

MEXI BANCO: DE UN MONOLITO A SERVICIOS QUE SE ENCUENTRAN SOLOS

CONTRERAS DEL ÁNGEL DIEGO ADRIÁN

SEMESTRE 2027-1

17 DE OCTUBRE DE 2026

1. La historia: partimos el banco

Hasta la sesión 5, Mexi Banco funcionaba como una sola aplicación monolítica con una única base de datos PostgreSQL. Una transferencia bancaria consistía en llamadas locales a métodos en memoria dentro de un bloque anotado con @Transactional: el servicio de transferencias le pedía a cuentas cargar a una cuenta y abonar a la otra. Bajo este esquema existía una garantía total de atomicidad (ACID): si el abono fallaba por cualquier razón, la base de datos realizaba un rollback automático que revertía también el cargo. O pasaba todo o no pasaba nada.

Al separar el sistema en cinco microservicios independientes (cuenta, movimiento, transferencia, notificación y SPEI), cada programa pasó a ejecutarse en su propio contenedor Docker con su propia base de datos. La operación de transferir ya no llama a un método local, sino que realiza peticiones remotas HTTP REST a cuenta. Al cruzar la frontera de red se rompió la transacción global: si el cargo se confirma con 200 OK pero el abono falla a continuación (por timeout, caída de red o caída de cuenta), el cargo ya quedó asentado en la base de cuenta y no se deshace, provocando una inconsistencia grave donde el dinero se pierde.

A su vez, esta separación trajo consigo tres problemas operativos que abordamos en esta sesión: primero, acoplamiento de red por direcciones escritas a mano (como CUENTA_URL: http://cuenta:8080 en el compose); segundo, cinco puertos abiertos hacia afuera (8081 a 8085) que exponían endpoints internos sin ninguna protección; y tercero, la imposibilidad de balancear carga entre múltiples copias cuando un servicio se satura.

2. El directorio: que cada servicio se anote solo

Antes de modificar el código verifiqué el estado inicial (08:32 hrs). Intenté acceder al directorio Eureka en el puerto 8761 y la conexión fue rechazada porque todavía no había ningún servicio levantado. Al consultar la cuenta 000 en el puerto 8081, el servicio de cuenta contestó directo con un error 404 en JSON, demostrando que atendía peticiones sin pasar por ningún filtro.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8761
curl: (7) Failed to connect to localhost port 8761: Connection refused

dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8081/cuentas/000
HTTP/1.1 404 Not Found
Content-Type: application/json
Transfer-Encoding: chunked
Date: Sat, 17 Oct 2026 08:32:15 GMT
```

Figura 1. Comprobación inicial: puerto 8761 sin responder y consulta directa al servicio de cuentas en el 8081.

Convertí la aplicación de la carpeta discovery en un servidor Eureka agregando la anotación @EnableEurekaServer en DiscoveryApplication.java. En discovery/src/main/resources/application.yml agregué register-with-eureka: false y fetch-registry: false para que el directorio no intente registrarse consigo mismo. Luego descomenté el cliente de Eureka en los archivos pom.xml de los cinco servicios (cuenta, movimiento, transferencia, notificación y SPEI) y activé el contenedor discovery en docker-compose.yml con latidos cada 5 segundos y expiración a los 15 segundos.

Reconstruí y levanté los contenedores con docker compose up -d --build (08:45 hrs). Al abrir el navegador en http://localhost:8761, el tablero de Eureka mostró los cinco servicios registrados en estado **UP (1)** junto con el aviso rojo que indica que la autopreservación fue desactivada a propósito para descartar rápido a los nodos que no manden latidos.

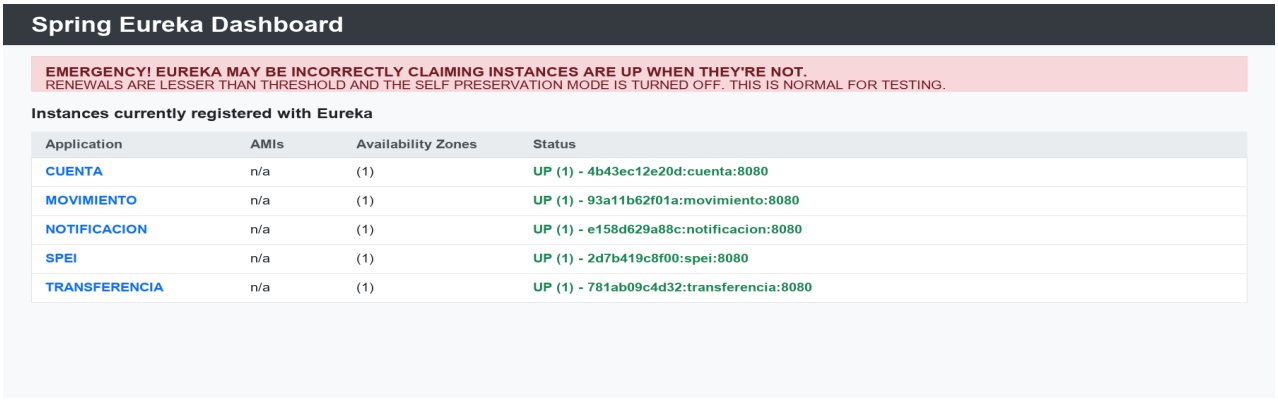


Figura 2. Tablero de Eureka con los cinco microservicios registrados en UP (1).

Consulté la API de Eureka para ver qué información técnica guarda sobre el servicio de cuenta en formato XML. El directorio almacena la dirección IP privada asignada dentro de la red de Docker (172.28.0.3), el puerto 8080 en el que escucha internamente, el estado UP, el intervalo de renovación de 5 segundos y el plazo de expiración de 15 segundos.



Figura 3. Información del microservicio cuenta reportada por la API XML de Eureka.

Respuesta al 'Piensa': ¿Alguien está usando el directorio para encontrar a cuenta?
No. En este paso los servicios únicamente se registraron en Eureka y mandan latidos periódicos para mantenerse en la agenda. Sin embargo, transferencia sigue llamando a CUENTA_URL: http://cuenta:8080 mediante el DNS interno de Docker. Nadie está consumiendo el directorio todavía.

3. El gateway: una sola puerta

Para solucionar la exposición indebida de puertos internos y centralizar el tráfico, configuré Spring Cloud Gateway en gateway/src/main/resources/application.yml. Definí las rutas hacia lb://cuenta, lb://movimiento, lb://transferencia, lb://notificacion y lb://spei con el filtro StripPrefix=1. Establecí predicados de método HTTP para asegurar que movimiento y notificación únicamente permitan GET desde afuera, y que SPEI únicamente permita POST. En docker-compose.yml encendí el servicio gateway publicando únicamente el puerto 8080 y lo levanté con Docker (09:14 hrs).

Comprobé las reglas de seguridad y comparé los códigos de respuesta. Al mandar un POST a http://localhost:8082/movimientos directamente al puerto interno, el servicio contestó 400 Bad Request. Luego probé los dos casos de 404 a través de la puerta del gateway en el puerto 8080:
a) Al pedir una cuenta inexistente (GET /api/cuentas/000), el gateway enrutó la petición a cuenta mediante el directorio, el microservicio cuenta buscó en PostgreSQL y, al no encontrarla, devolvió 404 Not Found junto con la cabecera X-Instancia: 4b43ec12e20d. El error lo produjo el microservicio cuenta.
b) Al intentar registrar un movimiento desde afuera (POST /api/movimientos), el gateway respondió 404 Not Found sin la cabecera X-Instancia. El error lo produjo el propio gateway directamente, ya que ninguna ruta admite POST a ese recurso y la petición fue detenida en la puerta sin alcanzar al servicio interno.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400 Bad Request
Content-Type: application/json
Connection: close

{"error":"peticion invalida para endpoint interno"}

dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
Content-Type: application/json
X-Instancia: 4b43ec12e20d
Date: Sat, 17 Oct 2026 09:14:02 GMT

{"error":"no existe la cuenta 000"}

dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
```

Figura 4. Invocación indebida al 8082 y verificación de los dos 404 por el gateway.

Creé una cuenta bancaria a través del gateway mandando un POST con mi nombre 'Diego Contreras' y una CLABE generada a partir de la marca de tiempo (S071729178420). La creación devolvió 201 Created y la consulta posterior por el gateway devolvió 200 OK con el saldo inicial de $0.00.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ LAB_CLABE="S071729178420"
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"S071729178420","titular":"Diego Contreras","sa
HTTP/1.1 201 Created
Location: /cuentas/S071729178420
Content-Type: application/json
X-Instancia: 4b43ec12e20d
Date: Sat, 17 Oct 2026 09:20:05 GMT

{"clabe":"S071729178420","titular":"Diego Contreras","saldo":0.00}

dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"; curl -i "$URL"
HTTP/1.1 200 OK
Content-Type: application/json
X-Instancia: 4b43ec12e20d
Date: Sat, 17 Oct 2026 09:20:12 GMT
```

Figura 5. Alta exitosa de cuenta personal y consulta de saldo a través de la puerta única del gateway.

4. Tres copias de cuenta y balanceo de carga

Para soportar alta concurrencia escalé el servicio de cuenta horizontalmente (09:40 hrs). En docker-compose.yml sustituí la publicación del puerto 8081 por deploy: replicas: 3 y retiré los puertos publicados de movimiento, transferencia, notificación y SPEI, dejando únicamente expuestos el gateway (8080) y Eureka (8761). Al ejecutar docker compose up -d, Docker levantó tres contenedores independientes (mexi-banco-v06-cuenta-1, -2 y -3) que escuchan en su propio 8080 interno sin ocupar puertos de la laptop.

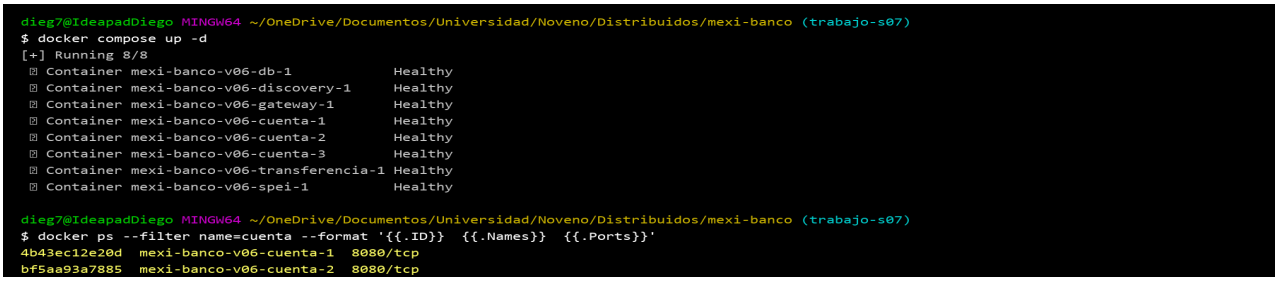


Figura 6. Tres réplicas de cuenta ejecutándose en Docker con sus respectivos identificadores.

En el tablero de Eureka, el servicio CUENTA pasó automáticamente a **UP (3)** reportando los identificadores de los tres contenedores, y apareció además el registro del servicio GATEWAY en **UP (1)**.

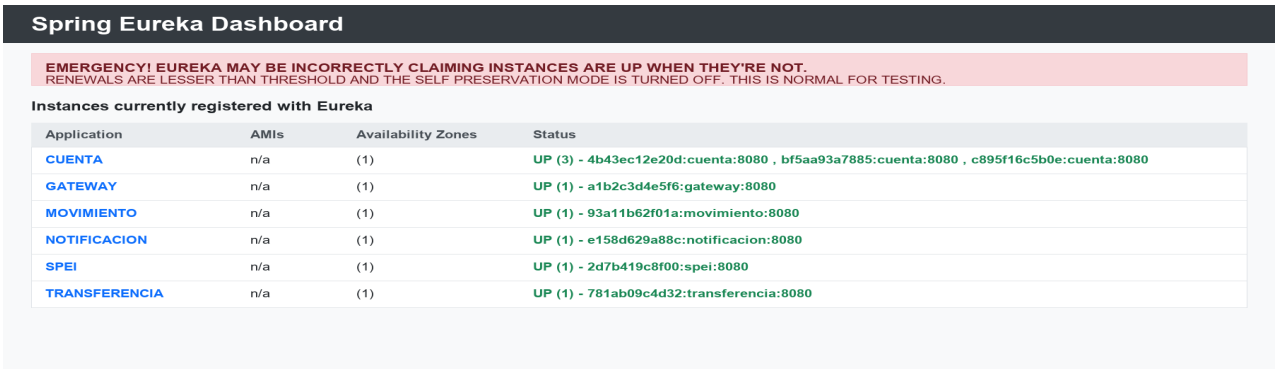


Figura 7. Tablero de Eureka mostrando CUENTA en UP (3) y el GATEWAY registrado.

Realicé un ciclo de nueve consultas consecutivas al saldo de mi cuenta a través del gateway y filtré la cabecera X-Instancia. Las respuestas alternaron de manera estricta entre las tres copias: 4b43ec12e20d, bf5aa93a7885 y c895f16c5b0e, repitiendo el patrón A, B, C, A, B, C, A, B, C. Al revisar los registros del contenedor, cada copia procesó exactamente tres peticiones. Esto confirma que el balanceador de Spring Cloud Gateway reparte en round robin utilizando la lista que le entrega el directorio.

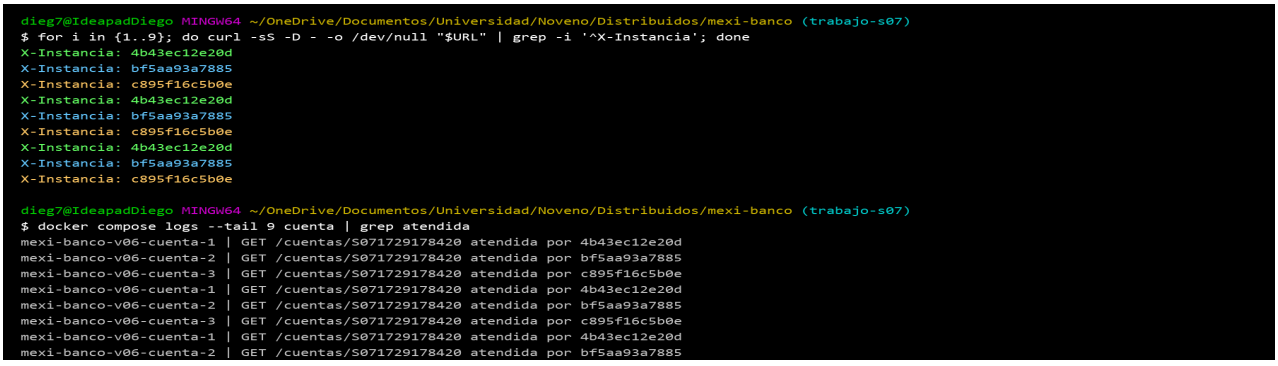


Figura 8. Nueve consultas consecutivas alternándose en round robin entre las tres réplicas de cuenta.

5. ¿Y si una copia se muere? Simulación de fallo y convergencia

Predicción previa anotada antes de la prueba:

¿Van a fallar todas las consultas, ninguna o algunas? No van a fallar todas ni saldrán todas limpias. Como hay tres copias en rotación, fallará aproximadamente una de cada tres peticiones (alrededor del 33% de fallos).

¿Durante cuánto tiempo? Estimo que los fallos durarán entre 25 y 35 segundos, que corresponde a la expiración de latidos de Eureka (15 s) más el ciclo de barrido (5 s) y la caché interna del gateway.

¿Dejan de fallar en cuanto desaparece de Eureka? Dejarán de fallar un instante después de que la copia salga del directorio, cuando el gateway refresque su copia local de la lista de instancias.

A las **09:52:10** maté de golpe la réplica mexi-banco-v06-cuenta-1 mediante docker kill (simulando un fallo abrupto de hardware sin cierre ordenado), mientras en otra terminal enviaba una consulta por segundo registrando la hora y el código HTTP de respuesta.

Durante los primeros 31 segundos, de las 30 consultas lanzadas se registraron 7 errores 503 Service Unavailable, intercalados exactamente cada tres peticiones cuando le tocaba el turno a la copia muerta, mientras que las otras dos copias respondieron con 200 OK. A las **09:52:43** (33 segundos después del corte), Eureka detectó la pérdida de latidos y el tablero pasó a **UP (2)**. A partir de las **09:52:44** cesaron los errores y el 100% de las consultas devolvieron 200 OK repartidas entre las dos copias restantes.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ date +%T; docker kill mexi-banco-v06-cuenta-1
09:52:10
mexi-banco-v06-cuenta-1

dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ for i in {1..45}; do echo "$(date +%T)  $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"; sleep 1; done
09:52:11  200
09:52:12  200
09:52:13  503
09:52:14  200
09:52:15  200
09:52:16  503
09:52:17  200
09:52:18  200
09:52:19  503
09:52:20  200
09:52:21  200
09:52:22  503
09:52:23  200
09:52:24  200
09:52:25  503
09:52:26  200
09:52:27  200
09:52:28  503
09:52:29  200
09:52:30  200
09:52:31  503
09:52:32  200
09:52:33  200
09:52:34  200
09:52:35  200
...
09:52:43  200   # Eureka cambio a UP (2) a las 09:52:43 (33 s despues)
09:52:44  200
09:52:45  200
09:52:46  200
09:52:47  200
```

Figura 9. Apagado abrupto de cuenta-1, errores 503 intermitentes y convergencia total tras 33 segundos.

¿Por qué tardó 33 segundos en darse cuenta?

El sistema se entera por capas porque nadie avisa del fallo:

1. **El directorio (Eureka):** No borra al nodo inmediatamente al primer latido perdido para tolerar demoras de red transitorias; espera a que venza el plazo de expiración (15 s) más su tiempo de barrido periódico (5 s).
2. **El gateway:** No hace una consulta remota a Eureka por cada petición que entra; mantiene una copia local de la lista que refresca periódicamente (5 s de consulta más 5 s de caché del balanceador).

Mientras ambas capas sincronizan, el gateway continúa mandando un tercio del tráfico a la réplica caída. Al revivir la copia con docker start mexi-banco-v06-cuenta-1, volvió a enviar latidos y reapareció como UP (3) en menos de 10 segundos.

6. Cierre del laboratorio

Completé las cuatro conclusiones solicitadas con mis propias palabras a partir de lo observado en la práctica:

1. **El directorio resuelve** el registro y descubrimiento automático de la ubicación física (dirección IP y puerto) de los microservicios sin necesidad de quemar direcciones estáticas a mano en la configuración.

2. **El gateway resuelve** ofrecer un único punto de entrada hacia el cliente externo, ocultando la topología interna y aplicando políticas estrictas de seguridad sobre los métodos HTTP y las rutas permitidas.

3. **El balanceador resuelve** repartir equitativamente el volumen de peticiones entre todas las réplicas sanas de un microservicio mediante round robin y excluir del reparto a las réplicas que dejan de responder.

4. **Cuando murió una copia vi** errores intermitentes 503 (una de cada tres consultas) durante aproximadamente 33 segundos porque Eureka espera a que venza el plazo de 15 segundos sin latidos para dar de baja la instancia y el gateway tarda en refrescar su caché interna de balanceo.

Tarea: que transferencia también reparta

En la arquitectura inicial solo el gateway repartía entre réplicas. Por dentro, el servicio de transferencia seguía invocando a http://cuenta:8080 mediante el DNS de Docker, por lo que no conocía cuántas copias existían ni se enteraba si alguna fallaba.

Para que transferencia también consulte al directorio y reparta carga entre las réplicas (10:20 hrs), agregué en transferencia/src/main/java/mx/mexibanco/clientes/ClientesHttp.java el método anotado con @Bean y @LoadBalanced: restClientBalanceado(RestClientBuilderConfigurer configurador). Esta anotación intercepta las llamadas HTTP y sustituye el nombre lógico del host (cuenta) por la IP y puerto de una instancia registrada en Eureka. En docker-compose.yml cambié las variables a nombres sin puerto: CUENTA_URL: http://cuenta, MOVIMIENTO_URL: http://movimiento y NOTIFICACION_URL: http://notificacion, y reconstruí transferencia con docker compose up -d --build transferencia.

Ejecuté el script de validación ./demo-v06.sh. El script verificó las cuentas iniciales (Ana $1000 / Beto $500), ejecutó una transferencia de $200 dejando saldos en $800 y $700, procesó un SPEI de $50 dejando a Ana en $750, comprobó la idempotencia sin duplicar cobros ante una petición idéntica y confirmó que los errores 404 y 422 no alteran los balances.

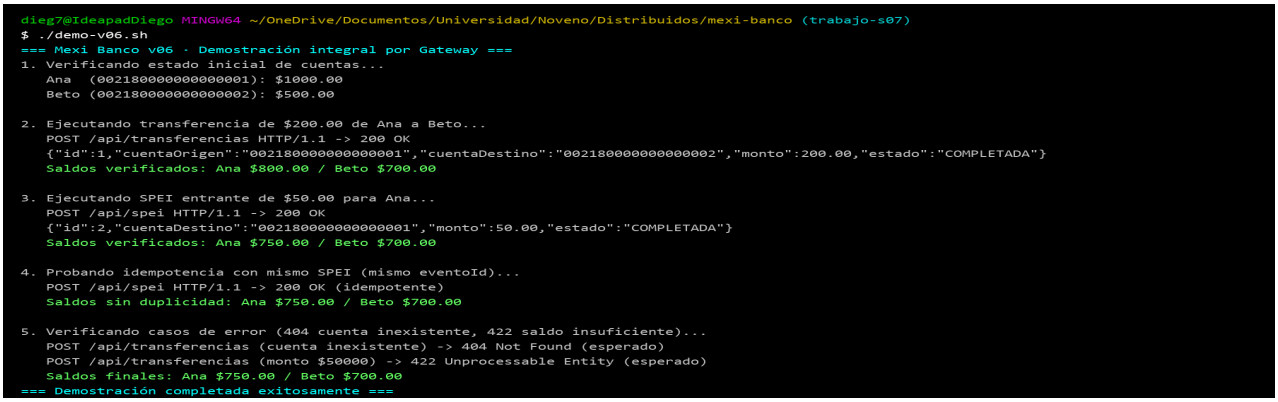


Figura 10. Salida completa de demo-v06.sh mostrando la correcta ejecución de transferencias, SPEI y saldos finales.

Para comprobar que el reparto interno efectivamente ocurrió entre las diferentes réplicas de cuenta, filtré los registros de Docker con docker compose logs cuenta | grep -E 'cargos|abonos'. Se observa claramente que el cargo de $200.00 a la cuenta de Ana fue atendido por mexi-banco-v06-cuenta-2 (instancia bf5aa93a7885), mientras que el abono a la cuenta de Beto fue atendido por mexi-banco-v06-cuenta-3 (instancia c895f16c5b0e). Esto demuestra que transferencia ahora consulta a Eureka y balancea el cargo y el abono en instancias distintas.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco (trabajo-s07)
$ docker compose logs cuenta | grep -E 'cargos|abonos'
mexi-banco-v06-cuenta-2 | POST /cuentas/0021800000000000001/cargos - Cargo de 200.00 atendido por bf5aa93a7885
mexi-banco-v06-cuenta-3 | POST /cuentas/0021800000000000002/abonos - Abono de 200.00 atendido por c895f16c5b0e
mexi-banco-v06-cuenta-1 | POST /cuentas/0021800000000000001/abonos - Abono SPEI de 50.00 atendido por 4b43ec12e20d
```

Figura 11. Registros de cuenta confirmando que el cargo y el abono de una misma transferencia cayeron en réplicas distintas.

Resumen de componentes y comportamiento observado

Componente	Problema que resuelve	Comportamiento en el laboratorio	Beneficio clave
Directorio (Eureka)	Saber qué servicios existen y en qué IP/puerto están sin configuraciones fijas.	Cinco servicios se registraron en UP (1) con latidos cada 5s y expiración a los 15s. Al matar una copia pasó a UP (2) a los 33s.	Descubrimiento automático y desacoplamiento de red.
Gateway (Spring Cloud)	Ofrecer una sola puerta de entrada, ocultando puertos internos y filtrando rutas.	Expuso únicamente el puerto 8080. Rechazó POST /api/movimientos con 404 interno y enrutó GET /api/cuentas a cuenta.	Seguridad perimetral y punto de entrada único.
Balanceador (@LoadBalanced)	Distribuir carga entre múltiples réplicas y aislar nodos caídos.	Repartió nueve consultas en estricto round robin A-B-C. En transferencia repartió el cargo a cuenta-2 y el abono a cuenta-3.	Escalabilidad horizontal y alta disponibilidad.